

High-Level Design (HLD)

Darviq-Buzz: A Social Platform Built as Microservices

*Polyglot persistence (PostgreSQL + Apache Cassandra), event-driven fan-out (RabbitMQ),
and real Prometheus + Alertmanager observability*

Document Title	Darviq-Buzz -- High-Level Design
Prepared For	Reference architecture
Prepared By	Ravi Shekhar Gurram
Version	1.0
Repository	D:/Projects/Social/Darviq-Buzz (local; not yet pushed to a remote)

Document Control

Solution Identification

Field	Value
Product / Solution Name	Darviq-Buzz
Sponsor / Context	Personal portfolio -- microservices, polyglot persistence, and observability practice
Author	Ravi Shekhar Gurram
Classification	Internal / Portfolio -- not for production use

Revision History

Version	Date	Author	Description
0.1	2026-09-20	Ravi Shekhar Gurram	Initial draft covering architecture, polyglot persistence, event backbone, and access management
1.0	2026-09-20	Ravi Shekhar Gurram	Finalized after live verification of the full user journey, fan-out-on-write, and alerting

Approval

Name	Role	Date	Signature
Ravi Shekhar Gurram	Author / Reviewer	2026-09-20	(self-reviewed -- portfolio project)

Table of Contents

Document Control	2
Solution Identification	2
Revision History	2
Approval	2
Table of Contents	3
References, Definitions, and Abbreviations	5
References	5
Definitions	5
Abbreviations	5
1. Overview	6
1.1 Background	6
1.2 Solution Summary	6
1.3 Scope of Document	6
1.4 Assumptions and Dependencies	6
1.5 Exclusions	6
2. Solution Architecture	7
2.1 Logical Architecture	7
2.1.1 Component Description	7
2.2 Interface Description	8
2.3 Deployment Architecture	8
2.4 Redundancy and Resilience	8
3. Data Architecture	9
3.1 Why Postgres and Cassandra Together	9
3.2 The Fan-Out-on-Write Feed	9
3.3 Data Model Summary	9
4. Access Management	11
4.1 Authentication	11
4.2 Trusted-Caller Pattern (web-bff)	11
4.3 External API Surface (gateway)	11
4.4 Network-Level Access Control	11
4.5 Least Privilege in Practice	11
5. Observability: Metrics and Alerts	12
5.1 Metrics	12

5.2 Alert Rules	12
5.3 Alertmanager.....	12
6. Scalability.....	13
6.1 What the Homelab Deployment Surfaced (real, not theoretical).....	13
7. Known Limitations and Future Work.....	14
8. Deliverables and Artifacts.....	15
8.1 Repository Layout.....	15
8.2 How to Run It.....	15
Appendix A: Service Inventory	16
Appendix B: Reaction Types	16
Appendix C: Verification Evidence.....	16

References, Definitions, and Abbreviations

References

Ref	Description
R1	Repository README.md -- architecture, running instructions, verification notes
R2	services/*/models.py -- per-service schema definitions (Postgres/Cassandra)
R3	prometheus/alerts.yml -- alert rule definitions
R4	k8s/network-policies.yaml -- NetworkPolicy definitions for the homelab deployment

Definitions

Term	Definition
Fan-out-on-write	Precomputing each follower's feed at post-creation time (one Timeline row per follower) instead of assembling it at read time
Partition read	A Cassandra query that reads a single partition key -- the cheapest, most scalable query shape in Cassandra
Trusted internal caller	A service (here, web-bff) that asserts an already-verified identity header directly, rather than re-verifying a JWT on every call
Business account	A User row with account_type=business plus category/contact fields -- reuses the same follow/post/feed machinery as a personal account

Abbreviations

Abbreviation	Meaning
BFF	Backend for Frontend
JWT	JSON Web Token
TTL	Time to Live
DM	Direct Message
SLO	Service Level Objective
FK	Foreign Key
CRUD	Create, Read, Update, Delete

1. Overview

1.1 Background

A previous single-Flask-app version of this project had genuinely well-designed domain logic (auth, feed assembly, follow/friend graphs, reactions, comments, reposts, hashtags, stories, DMs) but its templates, static assets, Helm chart, Terraform, Ansible, and tests were completely empty -- the app could not render a single page despite claiming to be verified end-to-end. This project is a from-scratch rebuild: the real domain logic was ported into a proper microservices architecture, and a genuinely working, browser-usable application was built and verified on top of it.

1.2 Solution Summary

Darvique-Buzz is an 11-service microservices application built to demonstrate polyglot persistence, event-driven architecture, and observability -- not merely "a working app." PostgreSQL backs the three services that need ad hoc filtering and simple read-after-write consistency on modest tables (users, notifications, media); Apache Cassandra backs the six services with genuinely partition-friendly, high-write social data (social graph, posts, engagement, stories, messaging, feed). RabbitMQ decouples the write path from its side effects: creating a post does not block on fanning it out to every follower's feed or notifying anyone who reacts to it.

1.3 Scope of Document

This HLD covers the full application: service decomposition, the polyglot persistence rationale, the event-driven fan-out-on-write feed architecture, access management between services, scalability characteristics, and the metrics/alerting layer. It reflects the application as verified locally via Docker Compose and, separately, as deployed to a homelab Kubernetes cluster (see Section 6 for what differs between the two).

1.4 Assumptions and Dependencies

- Docker and Docker Compose are available for local verification; a Kubernetes cluster (kind, in this project's case) is required for the homelab deployment path.
- No external identity provider is used -- user-service issues and verifies its own JWTs.
- RabbitMQ, Postgres, and Cassandra are each single-instance for this practice deployment; the architecture does not assume clustering for any of them.
- media-service's S3 upload path requires real AWS credentials to exercise; without them it deliberately and correctly falls back to local disk storage.

1.5 Exclusions

- A native mobile client -- the gateway's JWT-based API surface is designed for one, but none is built yet.
- Multi-region or multi-cluster deployment -- out of scope for this version.
- A production-grade CI/CD pipeline for the 11-service architecture -- the previous single-container design's Jenkinsfile/Helm/Terraform/Ansible are stale and explicitly deferred.

2. Solution Architecture

2.1 Logical Architecture

web-bff serves real, server-rendered pages directly to the browser and is a trusted internal caller of every other service -- it asserts an already-authenticated session's username via an X-User-Username header rather than re-verifying a JWT on every call, the same trust model a search-service calling a catalog-service directly would use. A separate gateway (JWT verification + reverse proxy) exists as the external-facing API surface for a future mobile or third-party client; web-bff does not go through it.

Figure 1: Service and Data Architecture Overview

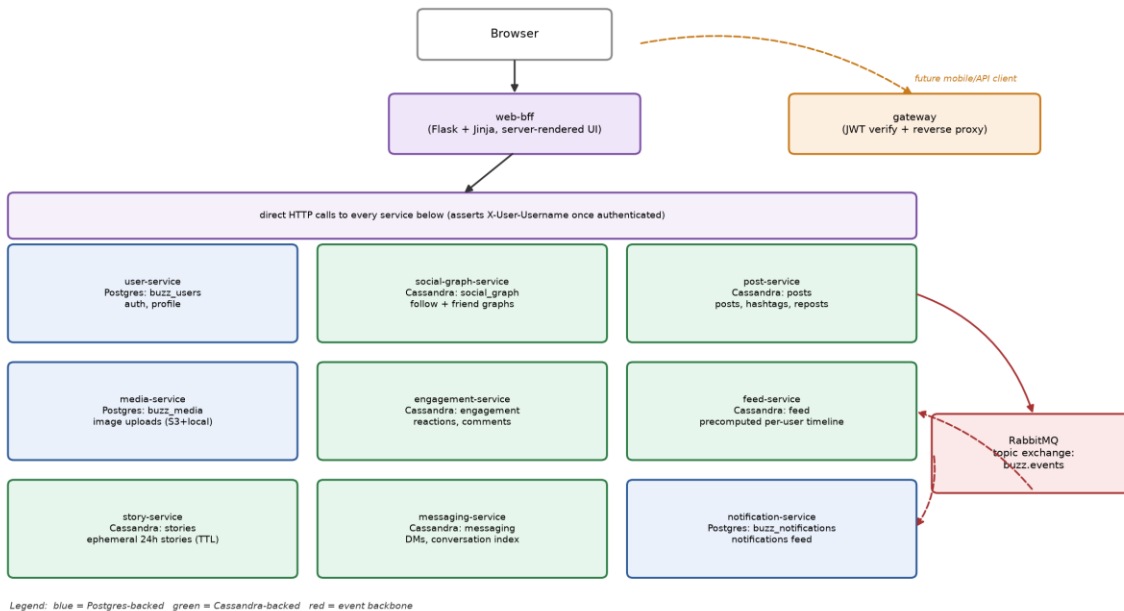


Figure 1: Service and data architecture overview

2.1.1 Component Description

2.1.1.1 web-bff

Flask + Jinja, real server-rendered UI (not a JSON API with a placeholder frontend). Holds the JWT in a signed server-side session cookie -- the browser never sees it -- and attaches it as an Authorization header on calls it makes on the user's behalf.

2.1.1.2 user-service, notification-service, media-service (Postgres)

These three own the data Cassandra is genuinely a poor fit for: a user's own profile row (read-after-write consistency matters immediately after signup/edit), a paginated notifications list with a read/unread flag, and a media asset registry that needs ad hoc lookups. This is a deliberate architectural split, not two databases for the sake of it -- see each service's models.py for the schema-level reasoning.

2.1.1.3 social-graph, post, engagement, story, messaging, feed-service (Cassandra)

Six services on genuinely partition-friendly, high-write social data: the follow and friend graphs, posts/hashtags/reposts, reactions and comments, ephemeral TTL-based stories, direct messages with a deterministic conversation-id index, and the precomputed per-user timeline. Every one of these is queried by a single, known partition key in the hot path -- exactly the shape Cassandra rewards.

2.1.1.4 RabbitMQ (topic exchange: buzz.events)

Decouples post-service (and others) from the side effects of a write: fanning a post out to every follower's feed, and notifying users of likes/comments/follows/friend requests. Each event is consumed independently by feed-service and notification-service; a slow or down consumer does not block the publisher.

2.2 Interface Description

Interface	Protocol	Direction	Purpose
Browser -> web-bff	HTTPS, server-rendered HTML	Client -> App	The actual application UI
web-bff -> every service	HTTP + X-User-Username header	Internal	Trusted-caller pattern, no re-verification of the JWT
Future client -> gateway	HTTPS + JWT Bearer token	External API	External-facing surface (mobile/3rd-party), not used by web-bff
post-service -> RabbitMQ	AMQP, topic exchange buzz.events	Publish	post.created, post.reposted, post.liked, post.commented
RabbitMQ -> feed-service / notification-service	AMQP consume	Subscribe	Fan-out-on-write and notification generation
Every service -> Prometheus	HTTP GET /metrics	Scrape	Request count + duration, plus service-specific metrics

2.3 Deployment Architecture

Two verified deployment paths exist for this application, exercising different parts of the stack:

Path	What it verifies
docker compose up -d --build (local)	The full 11-service stack, all 3 datastores, RabbitMQ, and Prometheus/Alertmanager, with the complete browser user journey exercised live
Homelab Kubernetes (kind)	Real container-orchestration concerns: resource limits, liveness/readiness probes, NetworkPolicies, in-cluster DNS, and rolling updates -- see Section 6 for the specific issues this surfaced and how they were fixed

2.4 Redundancy and Resilience

This is a single-replica practice deployment, not a highly-available production system. Resilience decisions made deliberately at this scale: RabbitMQ consumers ack/nack each message independently (one slow fan-out does not stall the whole consumer thread), and Cassandra's own replication (configured for a 3+ node cluster in a real deployment; single-node here) is what would provide durability at production scale, not application-level retry logic.

3. Data Architecture

3.1 Why Postgres and Cassandra Together

The split is drawn by actual data-access pattern, not an arbitrary rule of "some SQL, some NoSQL":

Need	Cassandra's fit	Postgres's fit
Ad hoc filtering / pagination on a modest table	Poor -- requires denormalizing or a secondary index workaround	Native
Immediate read-after-write consistency for one row	Eventually consistent by default at usable write levels	Native
Very high write volume, always queried by one known partition key	Excellent -- this is Cassandra's designed use case	Works, but doesn't scale writes the same way

user-service, notification-service, and media-service land in the left column of needs; social-graph-service, post-service, engagement-service, story-service, messaging-service, and feed-service land in the right.

3.2 The Fan-Out-on-Write Feed

The original design assembled a feed at read time: fetch every followed user's recent posts, merge, and sort in application code -- real, but it only scales to personal-app traffic. feed-service now precomputes the feed at write time instead.

Figure 2: Fan-Out-on-Write Feed -- Event Flow

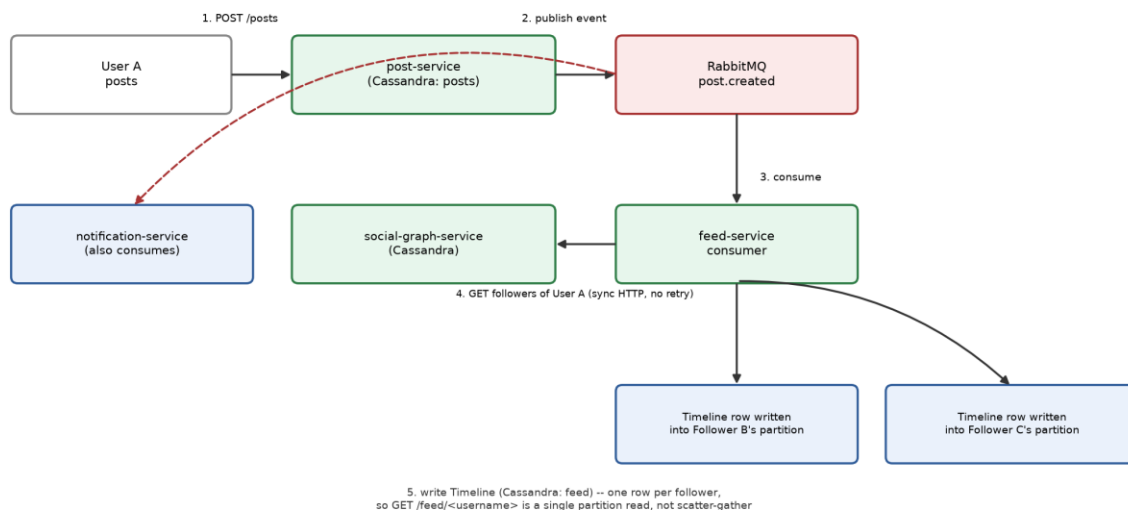


Figure 2: Fan-out-on-write feed -- event flow

post-service publishes post.created/post.reposted; feed-service consumes the event, looks up the author's followers via social-graph-service, and writes one Timeline row into every follower's own Cassandra partition. Reading a feed (GET /feed/<username>) is then a single partition read, not a scatter-gather across every followed user.

3.3 Data Model Summary

Service	Store	Key entities
---------	-------	--------------

user-service	Postgres: buzz_users	User (auth, profile, account_type, business fields)
social-graph-service	Cassandra: social_graph	Following/Followers, FriendRequest/Friendship
post-service	Cassandra: posts	Post, Hashtag, Repost
engagement-service	Cassandra: engagement	Like (6 reaction types), Comment
story-service	Cassandra: stories	Story (TTL-based ephemeral)
messaging-service	Cassandra: messaging	Message, ConversationIndex
feed-service	Cassandra: feed	Timeline (precomputed per-user feed)
notification-service	Postgres: buzz_notifications	Notification (recipient, actor, type, read)
media-service	Postgres: buzz_media	Media asset registry (bytes on S3 or local disk)

4. Access Management

4.1 Authentication

user-service issues and verifies its own JWTs (7-day expiry) at registration and login. Passwords are hashed with bcrypt (12 rounds) and are subject to a strict policy enforced both client-side (immediate feedback) and server-side (the actual authority): minimum 10 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.

4.2 Trusted-Caller Pattern (web-bff)

web-bff holds the session's JWT server-side and asserts the authenticated username via an X-User-Username header on every internal call, rather than re-verifying the JWT on every hop. Every backend service trusts this header implicitly because web-bff is the only caller reaching it directly on the internal network (enforced by Kubernetes NetworkPolicies in the homelab deployment -- see Section 4.4).

4.3 External API Surface (gateway)

gateway verifies the JWT itself (the caller here is untrusted, e.g. a future mobile app) and forwards the same trusted X-User-Username header downstream once verified -- the same header contract web-bff uses, so a service does not need to know or care which trusted caller it came from.

4.4 Network-Level Access Control

The homelab Kubernetes deployment adds Kubernetes NetworkPolicies restricting which pods may reach Cassandra, Postgres, and RabbitMQ -- for example, cassandra-allow-app-services permits ingress only from the specific services (and the one-shot keyspace-init Job) that actually need to reach it, denying every other pod in the namespace by default.

4.5 Least Privilege in Practice

A concrete bug found during the homelab deployment illustrates this being enforced for real, not just declared: the keyspace-init Job's pod had no app label, so it did not match the NetworkPolicy's selector even though the policy's author had already listed it as an intended allowed source -- its traffic was silently dropped until the Job's pod template was given the matching label. See Appendix C.

5. Observability: Metrics and Alerts

5.1 Metrics

Every service exposes GET /metrics in Prometheus format: request count and duration histograms labeled by service/route/status, plus service-specific metrics where they matter (feed-service's fan-out duration and follower-count histograms; media-service's upload duration and S3-vs-local-fallback counter).

5.2 Alert Rules

Alert	Trigger	Status
ServiceDown	up == 0 for 15s	Verified live: stopped a container and watched it fire within ~25s, then clear on restart, including the resolve transition
BuzzAvailabilitySLOFastBurn / SlowBurn	Multi-window burn-rate against a 99.9% availability SLO	Same pattern as the equivalent Nyaya-Setu alert
BuzzLatencySLOBreach	P99 latency < 1s SLO breach	Written, not live-triggered
FeedFanoutStalled / NotificationConsumerStalled	Async-pipeline health checks	Honestly documented as unable to distinguish "broken" from "genuinely quiet" at low traffic
MediaS3FallbackRateHigh	S3 fallback rate above threshold	Fires by default in local dev (no real AWS credentials) -- correct behavior, not a bug

5.3 Alertmanager

Routes every alert to a no-op default receiver -- enough to observe the firing/resolved lifecycle in Alertmanager's own UI without wiring a real Slack/email/PagerDuty target for a local practice stack.

6. Scalability

Growth dimension	What happens
More users/posts	Cassandra's partition-per-entity model (one partition per user's timeline, per conversation, etc.) scales horizontally by adding nodes -- no design change needed
More reads on the feed	Already $O(1)$ partition reads thanks to fan-out-on-write; read load does not grow with the number of accounts a user follows
More event volume	RabbitMQ consumers ack/nack per-message independently; feed-service and notification-service scale by adding consumer replicas, not by redesigning the queue
More services	The trusted-caller header pattern and per-service database ownership mean a new service integrates without touching existing ones' schemas

6.1 What the Homelab Deployment Surfaced (real, not theoretical)

Deploying to Kubernetes surfaced genuine resource-sizing and platform issues that a pure Docker Compose run never would have -- documented here because they are part of this system's real operational behavior, not swept under the rug:

- user-service's CPU limit (200m) was throttling bcrypt's deliberately CPU-hard password hashing to 2.2 seconds per hash; raised to 1000m dropped it to ~0.27s.
- Every Flask service ran its dev server single-threaded (no threaded=True), so a real request could queue behind a Kubernetes health-check probe hitting the same pod.
- An intermittent in-cluster DNS resolution failure (a known glibc-resolver race under load) caused sporadic "service unavailable" errors; fixed with the standard Kubernetes-documented mitigation (dnsConfig.options: single-request-reopen) plus a short application-level retry as a second line of defense.

7. Known Limitations and Future Work

- feed-service's synchronous call to social-graph-service (inside the RabbitMQ consumer) has no retry or circuit breaker -- a slow/down social-graph-service stalls that one fan-out, not the whole consumer, but the fan-out itself is not retried.
- No cross-service foreign keys (e.g. a notification's recipient_username is not a real FK into user-service's table) -- a deleted user leaves orphaned rows elsewhere. A real microservices tradeoff, not a bug to fix with a shared database.
- user.followed/friend.request_* style events are fire-and-forget with no transactional outbox -- a down RabbitMQ at publish time means the event is simply never seen.
- The homelab k8s manifests, Jenkins pipeline, GitOps, and infra-as-code for this 11-service architecture are a deferred follow-on phase; the previous single-container design's equivalents are stale.

8. Deliverables and Artifacts

Repository: D:/Projects/Social/Darviiq-Buzz (git-tracked locally).

8.1 Repository Layout

Path	Contents
services/<name>/	11 Flask services, each with its own models.py, app.py, requirements.txt, Dockerfile
postgres/init.sql	Creates the 3 Postgres databases on first container start
k8s/	Homelab Kubernetes manifests: namespace, all 11 deployments/services, NetworkPolicies, Cassandra/RabbitMQ/Postgres
prometheus/alerts.yml	Alert rule definitions
alertmanager/alertmanager.yml	Alertmanager routing config
docker-compose.yml	Full local stack: all 3 datastores, RabbitMQ, all 11 services, Prometheus, Alertmanager

8.2 How to Run It

```
docker compose up -d --build
# App is at http://localhost:8000 -- sign up, everything works, no manual seeding
```

Appendix A: Service Inventory

Service	Port	Store
web-bff	8000	-- (session only)
gateway	5000	-- (JWT verify + proxy)
user-service	5001	Postgres
social-graph-service	5002	Cassandra
post-service	5003	Cassandra
engagement-service	5004	Cassandra
story-service	5005	Cassandra
messaging-service	5006	Cassandra
feed-service	5007	Cassandra
notification-service	5008	Postgres
media-service	5009	Postgres

Appendix B: Reaction Types

engagement-service supports 6 reaction types on a post (richer than a single generic "like"): like, love, haha, wow, sad, angry.

Appendix C: Verification Evidence

Layer	Status
Full user journey (Docker Compose)	Verified live via real HTTP requests with cookie sessions: register, login, post (text + image), react, comment, repost, follow, send/accept a friend request (confirmed the auto-follow side effect), post a story, send a DM, see a notification appear, mark all read
Fan-out-on-write	Verified live: posted as one user, confirmed the post appeared in a different, already-logged-in follower's precomputed feed without that follower's own action
ServiceDown alert	Verified live: stopped a container, watched the alert fire in Prometheus within ~25s, confirmed the resolve transition on restart
Homelab Kubernetes deployment	Verified live end-to-end after fixing 4 real bugs surfaced only by actually deploying: probe timeouts too short for cqlsh/rabbitmq-diagnostics, a NetworkPolicy label mismatch on the keyspace-init Job, bcrypt CPU throttling, and an intermittent in-cluster DNS resolution failure
Strict password policy	Verified live: a weak password is rejected with the exact list of missing requirements; a compliant one succeeds